

Arsitektur Sistem Komputer Modern

Menjelajahi Von Neumann, CPU, ISA, dan
Manajemen Memori



Arsitektur Von Neumann

Fondasi Komputer Modern

Arsitektur Von Neumann adalah desain yang menjadi basis hampir semua komputer saat ini. Konsep utamanya adalah Program Tersimpan (Stored-Program Concept) di mana instruksi dan data disimpan dalam memori yang sama.



- Unit Pemrosesan Pusat (CPU)
- Unit Memori
- Input/Output (I/O)

Komponen Inti CPU

Otak dari Sistem Central Processing Unit (CPU) bertanggung jawab untuk menjalankan instruksi program melalui operasi logika dan aritmatika. Terdiri dari:



**Arithmetic Logic Unit
(ALU):**

Melakukan perhitungan.



Control Unit (CU):
Mengatur lalu lintas data
dan sinkronisasi.



Register:
Memori internal
berkecepatan sangat
tinggi.

Siklus Fetch-Decode-Execute

Bagaimana Program Berjalan CPU bekerja dalam siklus berulang:



Mengenal Register CPU

Penyimpanan Tercepat Register adalah lokasi penyimpanan kecil di dalam CPU untuk akses instan:



Program Counter (PC)

Menyimpan alamat instruksi berikutnya.



Instruction Register (IR)

Menyimpan instruksi yang sedang diproses.



Accumulator (ACC)

Menyimpan hasil antara dari perhitungan.



General Purpose Registers

Untuk manipulasi data sementara.

Instruction Set Architecture (ISA)

Bahasa Antara Software dan Hardware

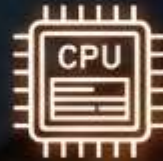
ISA adalah spesifikasi yang **mendefinisikan apa yang bisa dilakukan** oleh **CPU**.
Ini mencakup:



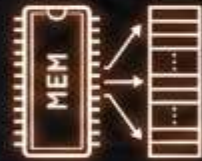
Set Instruksi
(Add, Move, Jump)



Tipe Data
yang didukung



Model Register



Manajemen memori virtual



ISA bertindak sebagai **kontrak** antara programmer (assembler/compiler) dan perangkat keras.



Klasifikasi ISA: RISC vs CISC

Dua Filosofi Desain

1. **RISC** (Reduced Instruction Set Computer): Instruksi sederhana, eksekusi cepat (Contoh: ARM, M1 Apple).



2. **CISC** (Complex Instruction Set Computer): Instruksi kompleks yang melakukan banyak tugas sekaligus (Contoh: Intel x86).



(Contoh: ARM, M1 Apple)



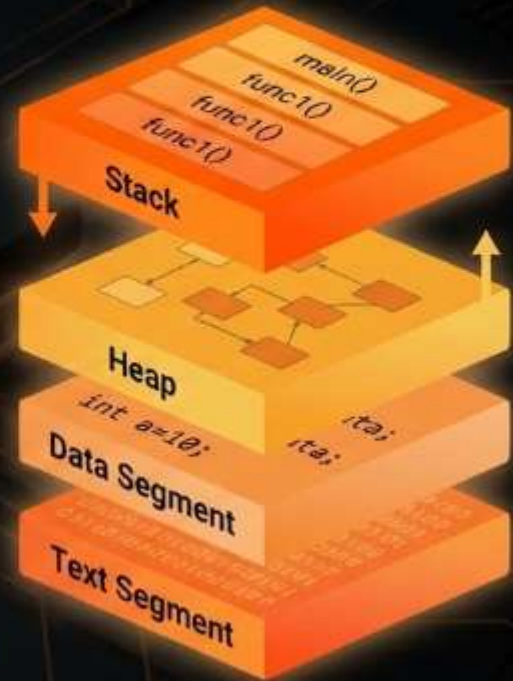
(Contoh: Intel x86)

Struktur Memori Program

Organisasi Ruang Alamat

Saat program dijalankan, sistem operasi mengalokasikan ruang memori yang terbagi menjadi beberapa segmen:

- ⚙️ **Text/Code Segment:** Instruksi biner program.
- 🗄️ **Data Segment:** Variabel global dan statis.
- 🗄️ **Stack:** Pemanggilan fungsi dan variabel lokal.
- 🏗️ **Heap:** Alokasi memori dinamis.



Data Segment: Statis dan Global

Penyimpanan Variabel Tetap Segmen data menyimpan informasi yang keberadaannya tetap selama program berjalan:

-  **Initialized Data:** Variabel global yang diberi nilai awal.
 -  **Uninitialized Data (BSS):** Variabel global yang belum diberi nilai awal.
- Data ini diletakkan di alamat memori yang tetap sebelum program mulai dieksekusi.

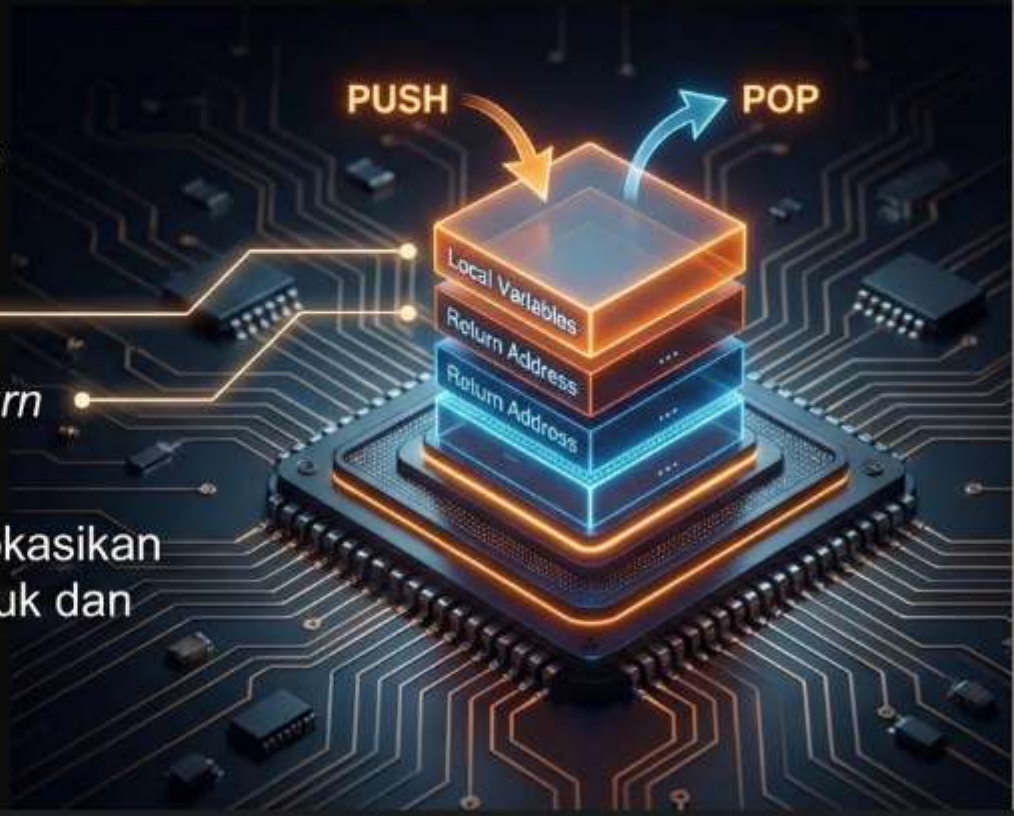


Stack Memory (Tumpukan)

Manajemen Fungsi LIFO

Memori Stack bekerja dengan prinsip Last-In-First-Out (LIFO):

- Menyimpan variabel lokal.
- Menyimpan alamat kembali (*return address*) saat fungsi dipanggil.
- Memori dialokasikan dan didealokasikan secara otomatis saat fungsi masuk dan keluar.



Heap Memory (Alokasi Dinamis)

Fleksibilitas Ruang Heap digunakan untuk data yang ukurannya tidak diketahui saat kompilasi:



Manual Allocation & Complex Structures

Dialokasikan secara manual oleh programmer (misal: malloc di C++). Memungkinkan pembuatan struktur data kompleks seperti linked list dan tree.

<https://www.petanikode.com/c-malloc/>

Management & Risks

Memerlukan manajemen manual atau Garbage Collection agar tidak terjadi memory leak.

<https://www.ibm.com/id-id/think/topics/garbage-collection-java>

PERAN SISTEM OPERASI (OS)

SANG KONDUKTOR SISTEM: PERANTARA EKSEKUSI PROGRAM



Process Management

Menjadwalkan CPU untuk berbagai program.



Memory Management

Mengatur isolasi memori antar proses melalui Virtual Memory.

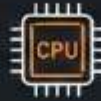


I/O Handling

Mengelola interaksi dengan hardware (disk, monitor).

KESIMPULAN: SINERGI SISTEM

Integrasi Total Eksekusi program yang efisien adalah hasil kerja sama antara:



1. **Arsitektur Hardware** (Von Neumann/CPU)



2. **Standar Instruksi (ISA)**



3. **Manajemen Memori yang Terstruktur (Stack/Heap)**



4. **Pengawasan Sistem Operasi.** Memahami lapisan ini adalah kunci bagi pengembangan perangkat lunak yang optimal.